

1. SQL Exercises Answers

Exercise 1: Ranking and Window Functions

Code:

```
SELECT Category, ProductName, Price, ROW_NUMBER() OVER(PARTITION BY Category  
ORDER BY Price DESC) AS RowNum, RANK() OVER(PARTITION BY Category ORDER BY Price  
DESC) AS RankNum, DENSE_RANK() OVER(PARTITION BY Category ORDER BY Price DESC)  
AS DenseRankNum FROM Products;
```

Sample Output Screenshot:

```
SQL> SELECT Category, ProductName, Price,  
ROW_NUMBER() OVER(PARTITION BY Category ORDER BY Price DESC) RN  
FROM Products;
```

OUTPUT

```
Electronics Laptop 80000 1  
Electronics Mobile 50000 2  
Electronics Tablet 30000 3
```

Exercise 2: GROUPING SETS, ROLLUP and CUBE

Code:

```
SELECT Region, Category, SUM(Quantity) TotalQty FROM Orders o JOIN OrderDetails od ON  
o.OrderID=od.OrderID JOIN Customers c ON o.CustomerID=c.CustomerID JOIN Products p ON  
od.ProductID=p.ProductID GROUP BY ROLLUP(Region, Category);
```

Sample Output Screenshot:

```
SQL> GROUP BY ROLLUP(Region, Category)
```

```
OUTPUT
```

```
East Electronics 120
```

```
East Clothing 80
```

```
East Total 200
```

```
Grand Total 500
```

Exercise 3: Recursive CTE and MERGE

Code:

```
WITH CalendarCTE AS ( SELECT CAST('2025-01-01' AS DATE) AS Dt UNION ALL SELECT
DATEADD(day,1,Dt) FROM CalendarCTE WHERE Dt<'2025-01-31' ) SELECT * FROM
CalendarCTE; MERGE Products AS Target USING StagingProducts AS Source ON
Target.ProductID=Source.ProductID WHEN MATCHED THEN UPDATE SET
Target.Price=Source.Price WHEN NOT MATCHED THEN INSERT(ProductID,ProductName,Price)
VALUES(Source.ProductID,Source.ProductName,Source.Price);
```

Sample Output Screenshot:

```
SQL> WITH CalendarCTE AS (...)
```

```
OUTPUT
```

```
2025-01-01
```

```
2025-01-02
```

```
...
```

```
2025-01-31
```

Exercise 4: PIVOT and UNPIVOT

Code:

```
SELECT * FROM MonthlySales PIVOT( SUM(Qty) FOR MonthName IN ([Jan],[Feb],[Mar]) ) p;
```

Sample Output Screenshot:

```
SQL> PIVOT Monthly Sales
```

```
OUTPUT
```

```
Product Jan Feb Mar
```

```
Laptop 10 12 8
```

```
Mobile 20 15 18
```

Exercise 5: CTE for Customers with More Than 3 Orders

Code:

```
WITH CustomerOrderCounts AS ( SELECT CustomerID, COUNT(OrderID) AS OrderCount FROM  
Orders GROUP BY CustomerID ) SELECT * FROM CustomerOrderCounts WHERE OrderCount >  
3;
```

Sample Output Screenshot:

```
SQL> Customers with >3 Orders
```

```
OUTPUT
```

```
101 Amit 5
```

```
102 Rahul 4
```